

Linear Search

```
int *data;
int key;
int main()
{
    int number = 3;
    data = new int[ number ];

    for( int i = 0; i < number; i++ )
        data[ i ] = 1 + rand() % 9;

    cout << "\n\nEnter the integer search key: ";
    cin >> key;

    int position = linearSearch( number - 1 );

    if( position != -1 )
        cout << "\nFound key in position " << position << ".\n\n";
    else
        cout << "\nKey not found.\n\n";

    delete[] data;
}
```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; i++ )
        if( data[ i ] == key )
            return i;
    return -1;
}
```

Recursive Linear Search

```
int linearSearch( int last )  
{
```

```
}
```

```
int linearSearch( int last )  
{  
    for( int i = 0; i <= last; i++ )  
        if( data[ i ] == key )  
            return i;  
    return -1;  
}
```

```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;
}
```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; i++ )
        if( data[ i ] == key )
            return i;
    return -1;
}
```

```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

}
```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; i++ )
        if( data[ i ] == key )
            return i;
    return -1;
}
```

```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch(          );
}
```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; i++ )
        if( data[ i ] == key )
            return i;
    return -1;
}
```



```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; i++ )
        if( data[ i ] == key )
            return i;
    return -1;
}
```

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

data

0	1	2
6	9	8

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
                                2

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

data

0	1	2
6	9	8

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

key

6

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

key

6

data

0	1	2
6	9	8


```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

6

data

0	1	2
6	9	8


```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

data

0	1	2
6	9	8

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
                                2

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

data

0	1	2
6	9	8

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

data

0	1	2
6	9	8


```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

-1

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

key

4

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

-1

1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

-1

0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

0

key

4

data

0	1	2
6	9	8

```

int main()
{
    int number = 3;
    int position = linearSearch( number - 1 );
}

```

-1 2

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

-1 1

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

-1 0

```

int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key ) return 0;
        else return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
}

```

key

4

data

0	1	2
6	9	8

```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```
int linearSearch( int last )
{
    if( last == -1 )
        return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```

int data[ size ];
int main() 3
{
    int position = linearSearch( size - 1 );
                                0                2

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
                                0                1

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
                                0                0

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;

```

key

6

data

0	1	2
6	9	8

```

int data[ size ];
int main() 3
{
    int position = linearSearch( size - 1 );
                                -1          2

```

```

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
                                -1          1

```

```

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
                                -1          0

```

```

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;
    return linearSearch( last - 1 );
                                -1          -1

```

```

int linearSearch( int last )
{
    if( last == -1 ) return -1;
    if( data[ last ] == key ) return last;

```

key

4

data

0	1	2
6	9	8


```
int *data;
int key;
int main()
{
    int number = 8;
    data = new int[ number ];

    for( int i = 0; i < number; ++i )
        data[ i ] = 10 + rand() % 90;

    for( int i = 0; i < number; ++i )
        cout << setw( 3 ) << data[ i ];
    cout << endl << endl;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( number - 1 );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n";
    delete[] data;
}
```

```

int *data;
int key;
int main()
{
    int number = 8;
    data = new int[ number ];

    for( int *p =      ; p <      ; ++p )
        *p = 10 + rand() % 90;

    for( int *p =      ; p <      ; ++p )
        cout << setw( 3 ) << *p << " ";
    cout << endl << endl;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( number - 1 );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n";
    delete[] data;
}

```

```
int *data;
int key;
int main()
{
    int number = 8;
    data = new int[ number ];

    for( int *p = data; p < data + number; ++p )
        *p = 10 + rand() % 90;

    for( int *p = data; p < data + number; ++p )
        cout << setw( 3 ) << *p;
    cout << endl << endl;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( number - 1 );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n";
    delete[] data;
}
```

```

int linearSearch( int last )
{
    for( int i = 0; i <= last; ++i )
        if( data[ i ] == key )
            return i;
    return -1;
}

```

```

int* linearSearch( int last )
{
    for( int *p =      ; p <=      ;      )
        if(      == key )
            return p;
    return nullptr;
}

```

```
int linearSearch( int last )
{
    for( int i = 0; i <= last; ++i )
        if( data[ i ] == key )
            return i;
    return -1;
}
```

```
int* linearSearch( int last )
{
    for( int *p = data; p <= data + last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}
```

```

int *data;  int key;      data       key 
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( number - 1 );
}
 
      position

```

```

int* linearSearch( int last )
{
    for( int *p = data; p <= data + last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

```
int *data;  int key;
```

data 0012FF70

key 44

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

```
int *data;  int key;      data 0012FF70      key 44
```

```
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( number - 1 );
}
```



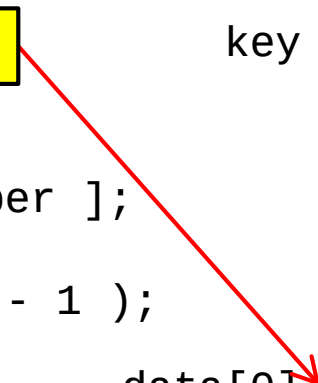
position



last 7

```
int* linearSearch( int last )
{
    for( int *p = data; p <= data + last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}
```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90




```
int *data;  int key;
```

data 0012FF70

key 44

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



p

0012FF70

last

7

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

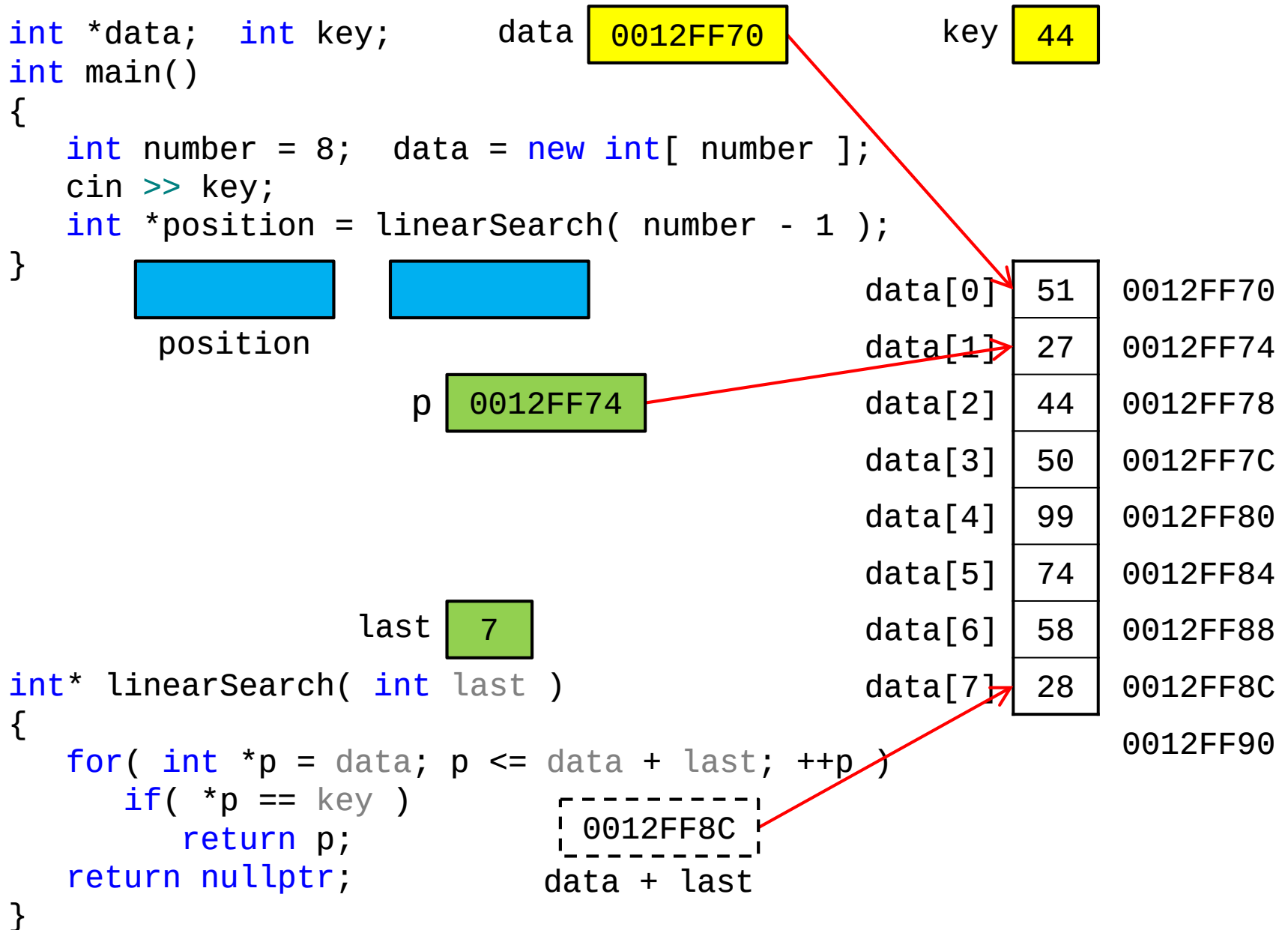
```
    return nullptr;
```

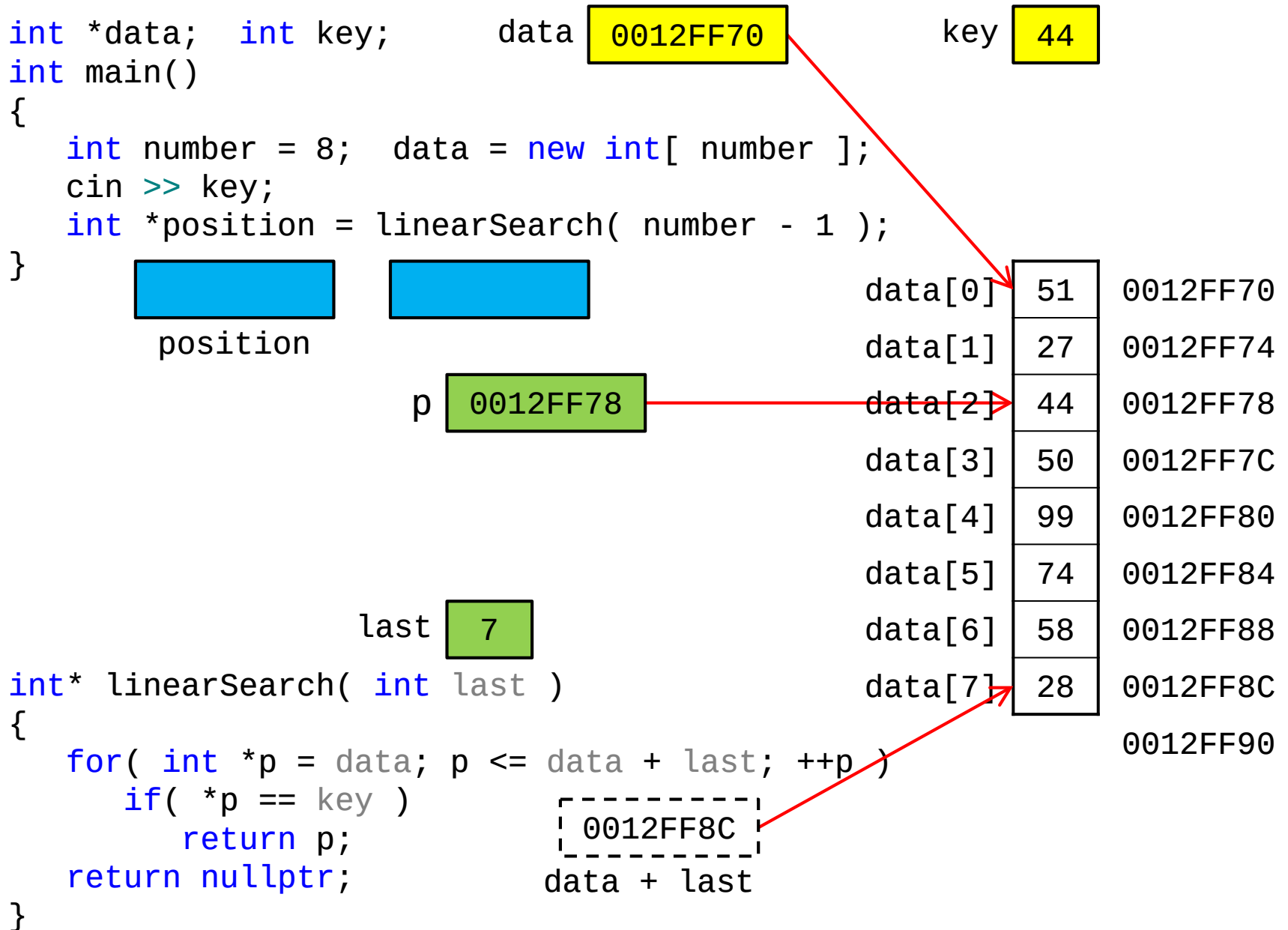
```
}
```

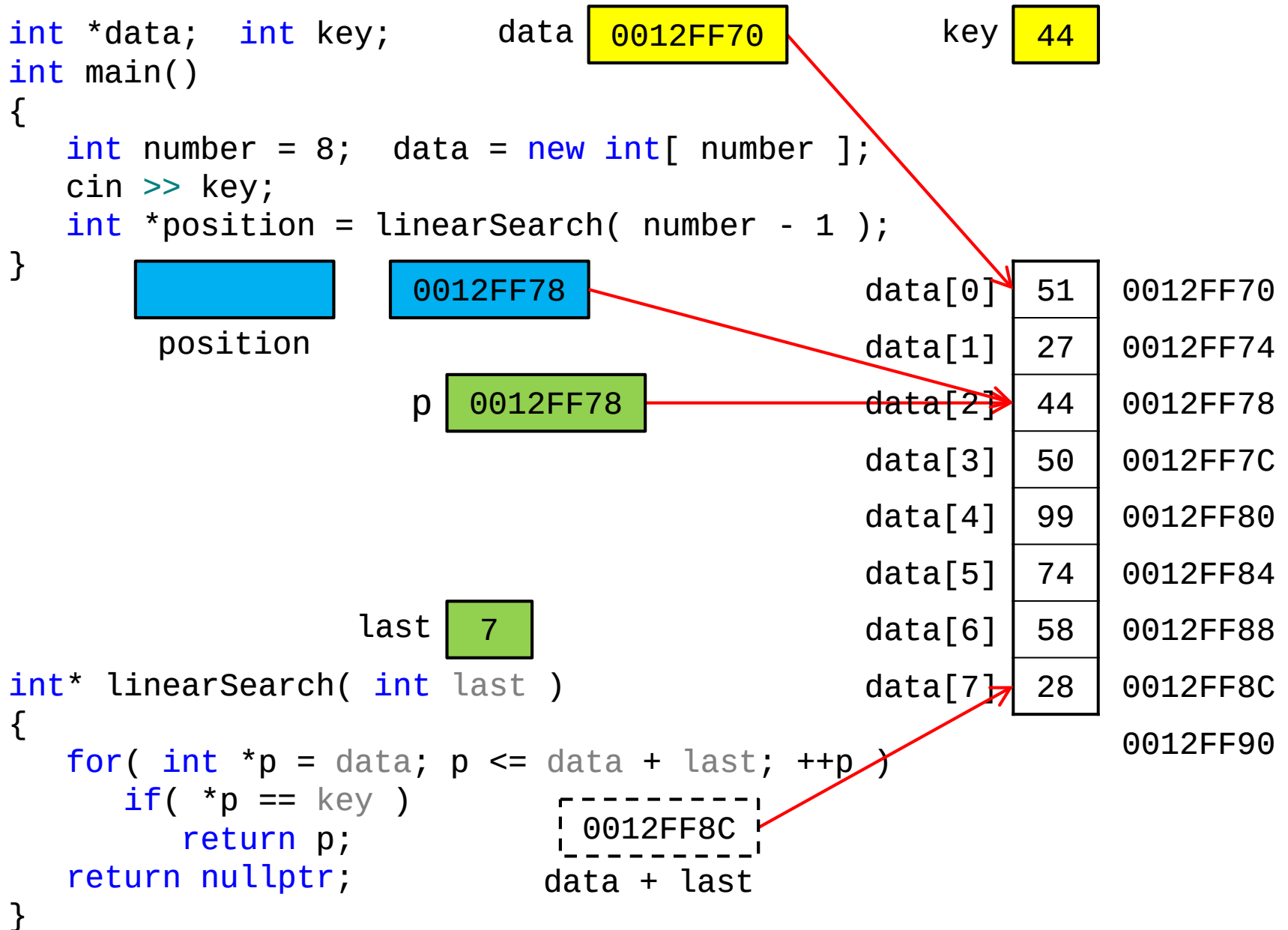
data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

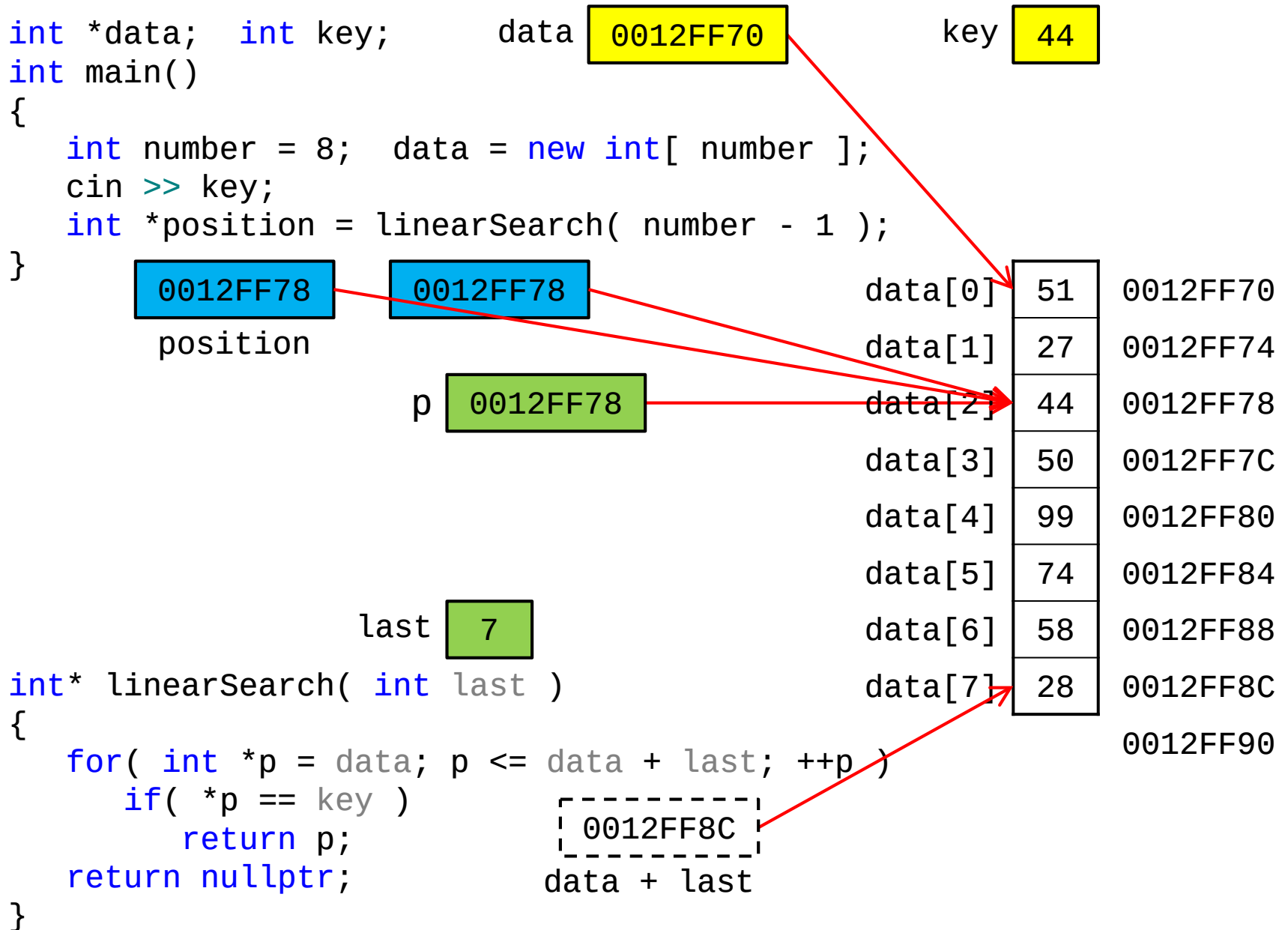
0012FF8C

data + last









```
int *data;
int key;
int main()
{
    int number = 8;
    data = new int[ number ];

    for( int *p = data; p < data + number; ++p )
        *p = 10 + rand() % 90;

    for( int *p = data; p < data + number; ++p )
        cout << setw( 3 ) << *p;
    cout << endl << endl;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( number - 1 );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n";
    delete[] data;
}
```



```

int *data;  int key;      data       key 
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( number - 1 );
}
 
      position

```

```

int* linearSearch( int last )
{
    for( int *p = data; p <= data + last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```



```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



p



last



```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



p

0012FF70

last

7

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + last

```
int *data;  int key;      data 0012FF70      key 55
```

```
int main()  
{  
    int number = 8;  data = new int[ number ];  
    cin >> key;  
    int *position = linearSearch( number - 1 );  
}
```



position



p 0012FF74

last 7

```
int* linearSearch( int last )  
{  
    for( int *p = data; p <= data + last; ++p )  
        if( *p == key )  
            return p;  
    return nullptr;  
}
```

0012FF8C

data + last

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

```
int *data;  int key;      data 0012FF70      key 55
```

```
int main()  
{  
    int number = 8;  data = new int[ number ];  
    cin >> key;  
    int *position = linearSearch( number - 1 );  
}
```

position

p 0012FF88

last 7

```
int* linearSearch( int last )  
{  
    for( int *p = data; p <= data + last; ++p )  
        if( *p == key )  
            return p;  
    return nullptr;  
}
```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C
data + last

```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

p 0012FF8C

last 7

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

0012FF8C

data + last

```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```



position



data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

p 0012FF90

last 7

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

0012FF8C

data + last

```
int *data;  int key;
```

data 0012FF70

key 55

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```

position

0

data[0] 51 0012FF70

data[1] 27 0012FF74

data[2] 44 0012FF78

data[3] 50 0012FF7C

data[4] 99 0012FF80

data[5] 74 0012FF84

data[6] 58 0012FF88

data[7] 28 0012FF8C

0012FF90

p 0012FF90

last 7

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

0012FF8C

data + last


```
int *data;  int key;
```

data **0012FF70**

key **55**

```
int main()
```

```
{
```

```
    int number = 8;  data = new int[ number ];
```

```
    cin >> key;
```

```
    int *position = linearSearch( number - 1 );
```

```
}
```

0

position

0

p **0012FF90**

last **7**

```
int* linearSearch( int last )
```

```
{
```

```
    for( int *p = data; p <= data + last; ++p )
```

```
        if( *p == key )
```

```
            return p;
```

```
    return nullptr;
```

```
}
```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + last

```
int *data;
int key;
int main()
{
    int number = 8;
    data = new int[ number ];

    for( int *p = data; p < data + number; ++p )
        *p = 10 + rand() % 90;

    for( int *p = data; p < data + number; ++p )
        cout << setw( 3 ) << *p;
    cout << endl << endl;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( number - 1 );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n";
    delete[] data;
}
```

```
int* linearSearch( int last )  
{  
    for( int *p = data; p <= data + last; ++p )  
        if( *p == key )  
            return p;  
    return nullptr;  
}
```

```
int *data;
int key;
int main()
{
    srand( static_cast< int >( time( 0 ) ) );
    int number = 20;
    data = new int[ number ];

    for( int *p = data; p < data + number; ++p )
        *p = 10 + rand() % 90;

    cout << "Enter search key: ";
    cin >> key;

    int *position = linearSearch( data + ( number - 1 ) );

    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n\n";

    delete[] data;
}
```

```
int* linearSearch( int      )
{
    for( int *p = data; p <=      ; ++p )
        if( *p == key )
            return p;
    return nullptr;
}
```

```
int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}
```

```

int *data;  int key;      data       key 
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}
 
    position

```

```

int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)

```

int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```



```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



last 0012FF8C

```

int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)

```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



p 0012FF70

last 0012FF8C

```

int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)

```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



p 0012FF74

last 0012FF8C

```

int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)

```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



p 0012FF88

last 0012FF8C

```

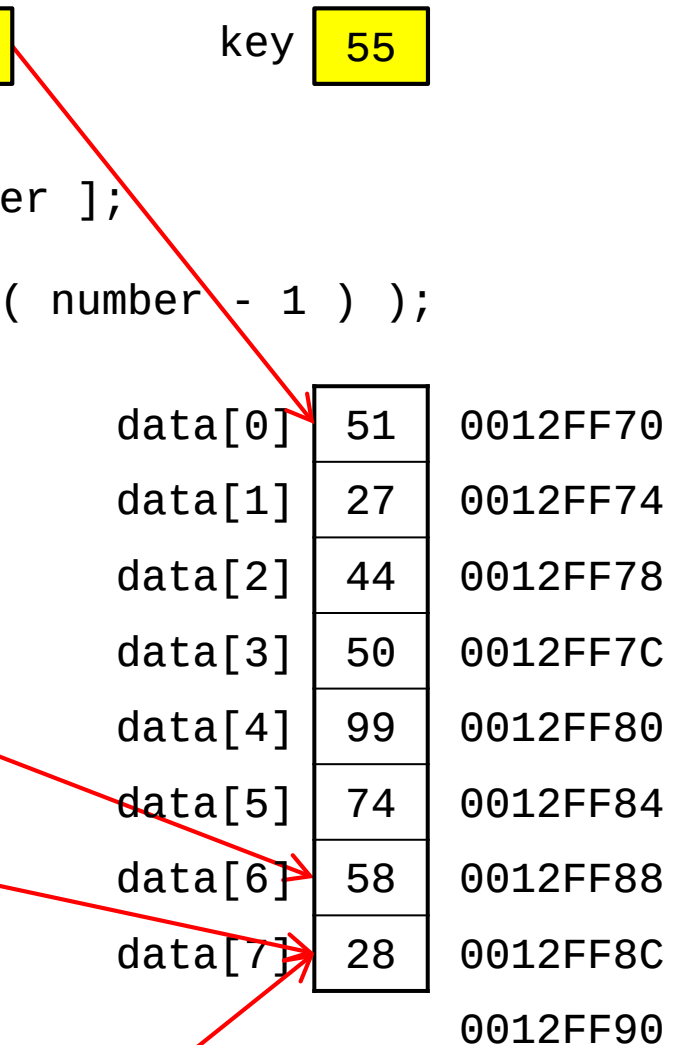
int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)



```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



p 0012FF8C

last 0012FF8C

```

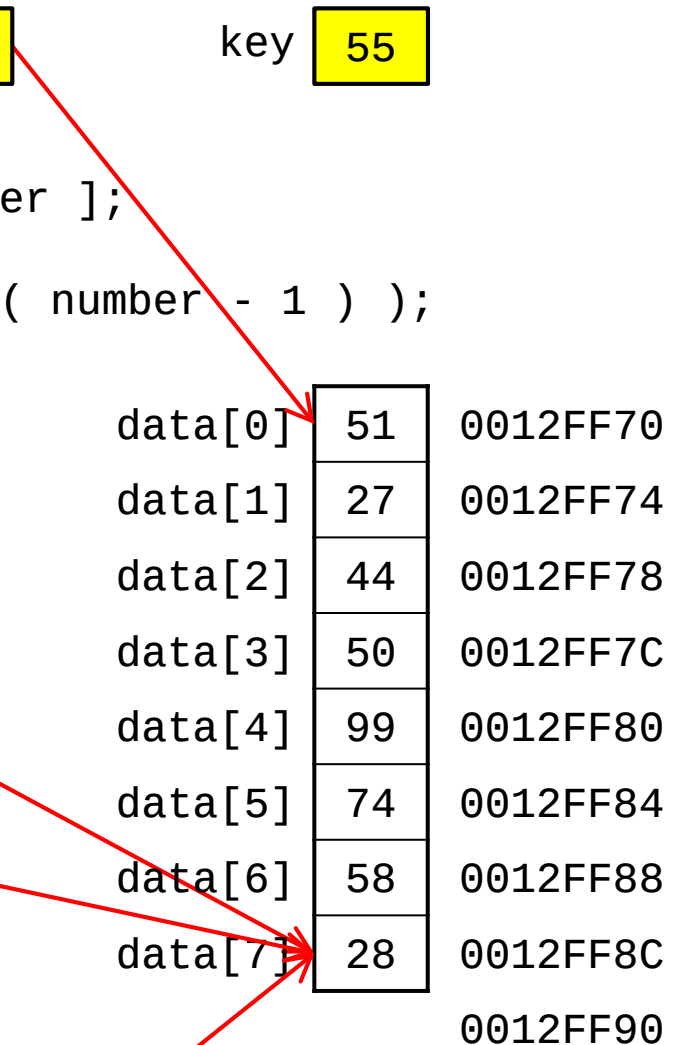
int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)



```

int *data;  int key;      data 0012FF70      key 55
int main()
{
    int number = 8;  data = new int[ number ];
    cin >> key;
    int *position = linearSearch( data + ( number - 1 ) );
}

```



position



p 0012FF90

last 0012FF8C

```

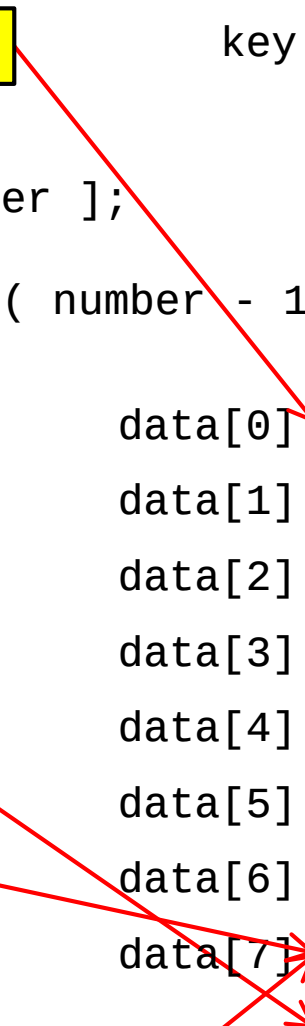
int* linearSearch( int *last )
{
    for( int *p = data; p <= last; ++p )
        if( *p == key )
            return p;
    return nullptr;
}

```

data[0]	51	0012FF70
data[1]	27	0012FF74
data[2]	44	0012FF78
data[3]	50	0012FF7C
data[4]	99	0012FF80
data[5]	74	0012FF84
data[6]	58	0012FF88
data[7]	28	0012FF8C
		0012FF90

0012FF8C

data + (number - 1)



Binary Search

```
int main()
{
    srand( static_cast< int >( time( 0 ) ) );

    int number = 15;
    data = new int[ number ];

    for( int i = 0; i < number; i++ )
        data[ i ] = 10 + rand() % 90;

    bubbleSort( number );

    cout << "Enter search key: ";
    cin >> key;

    int position = binarySearch( 0, number - 1 );
    if( position != -1 )
        cout << "Found key in position " << position << ".\n\n";
    else
        cout << "Key not found.\n\n";

    delete[] data;
}
```



```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```

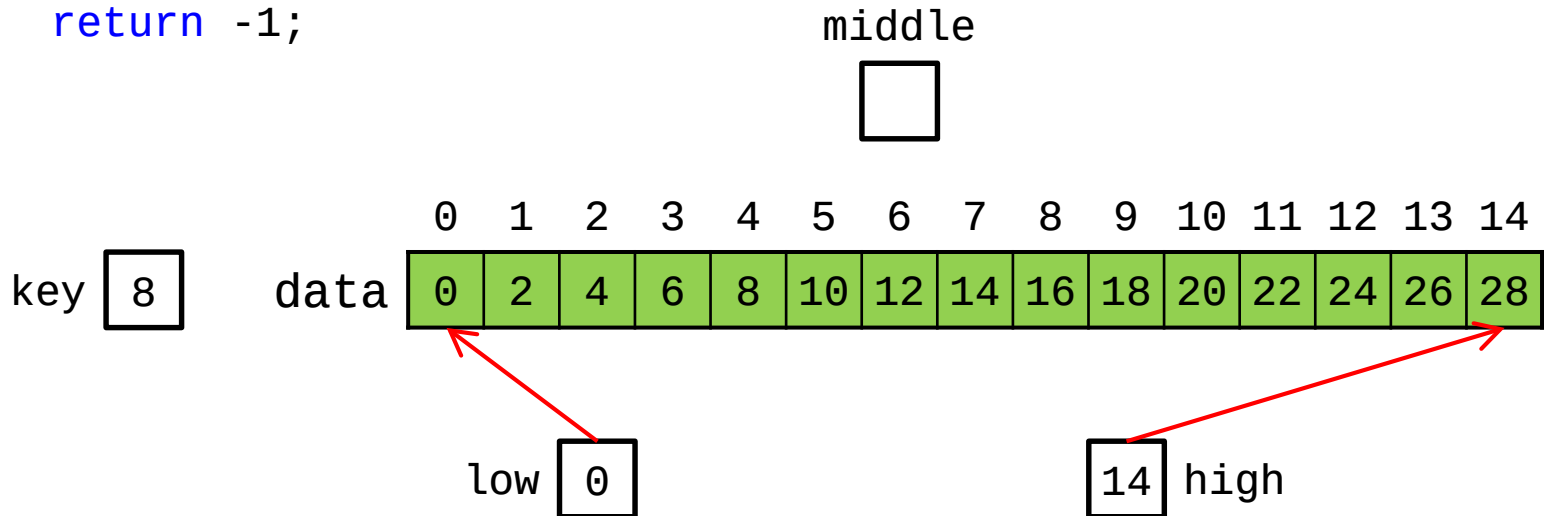
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



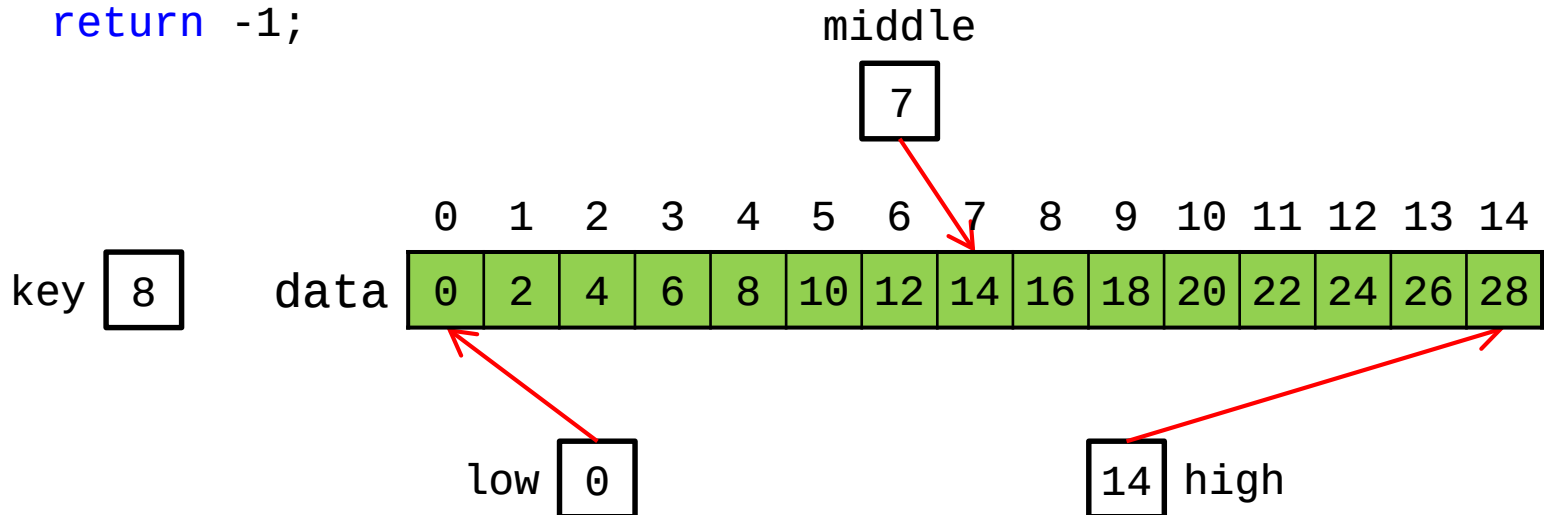
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



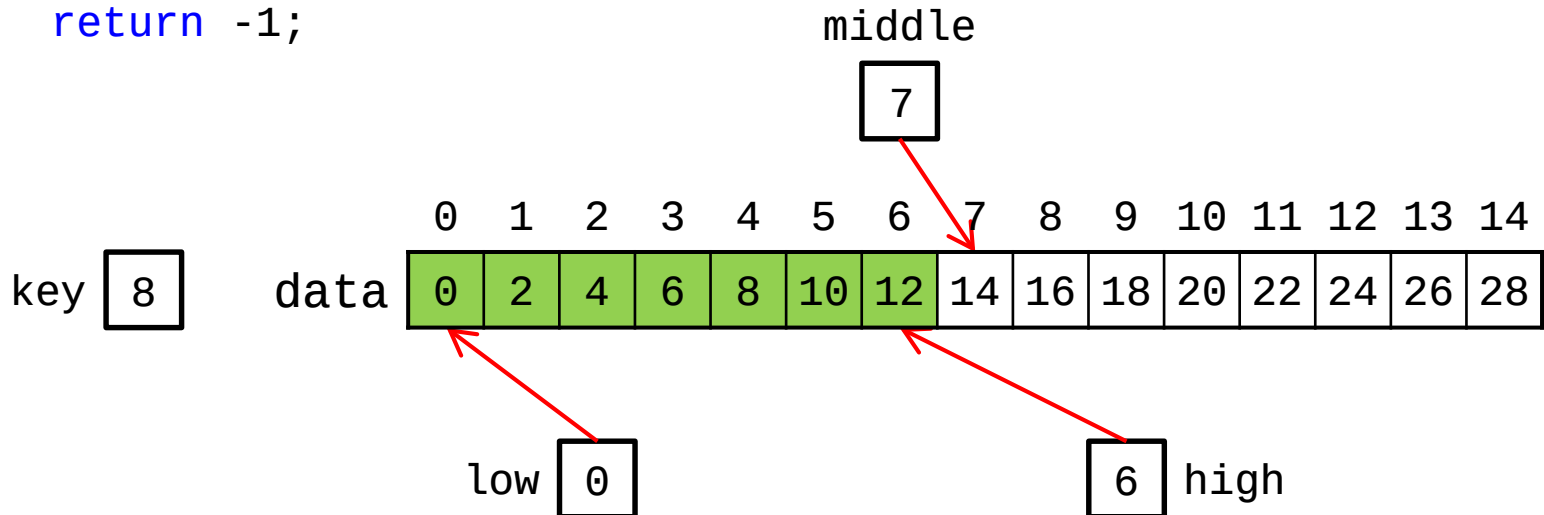
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



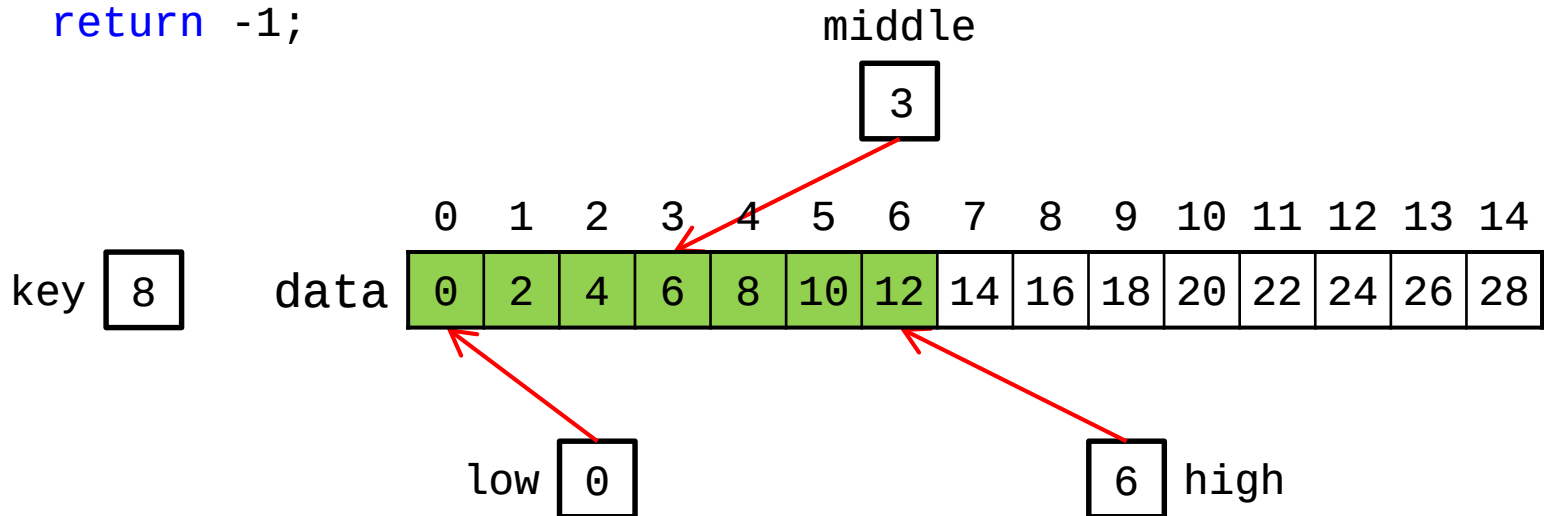
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



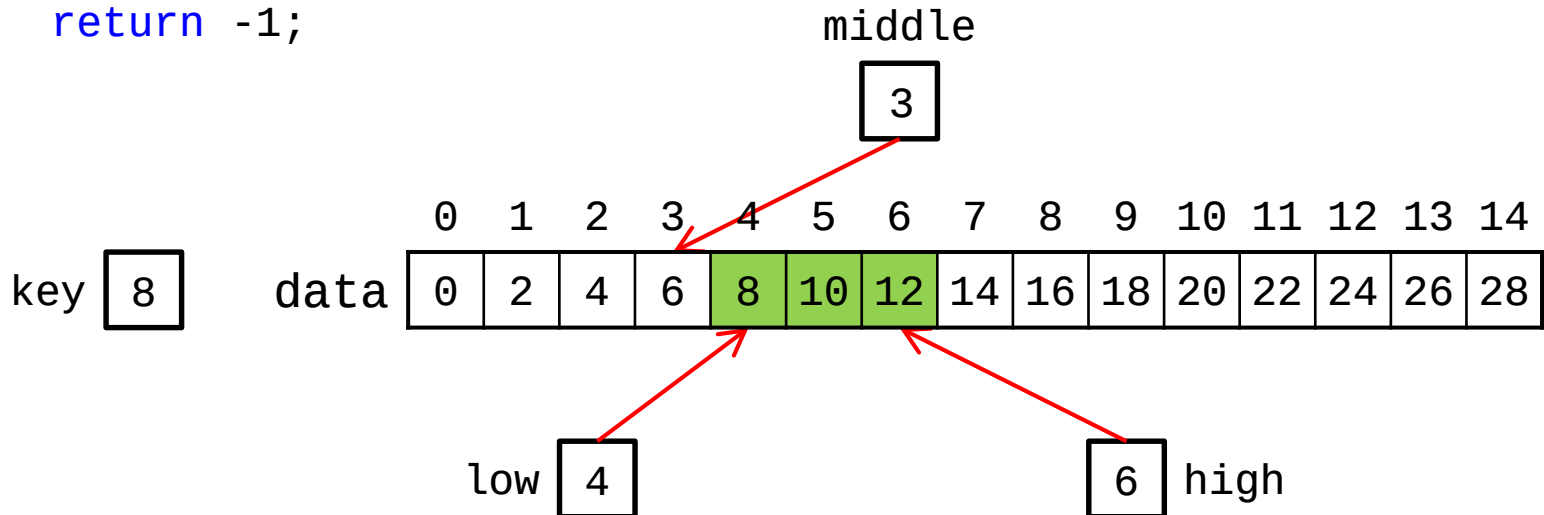
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



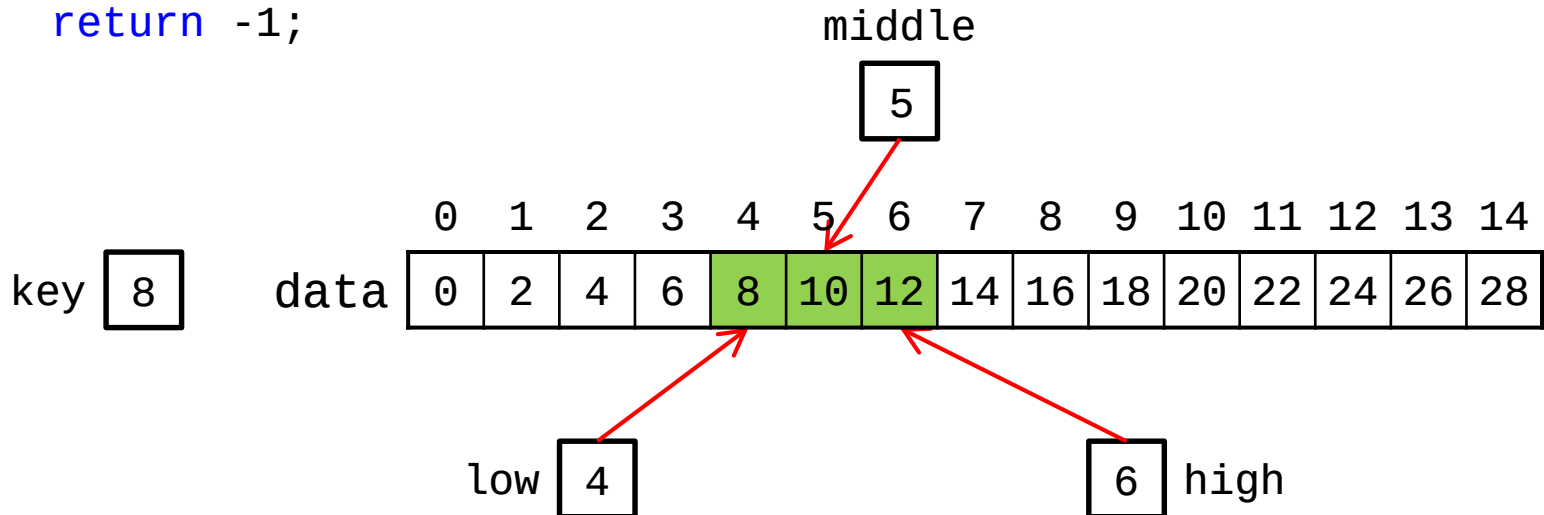
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



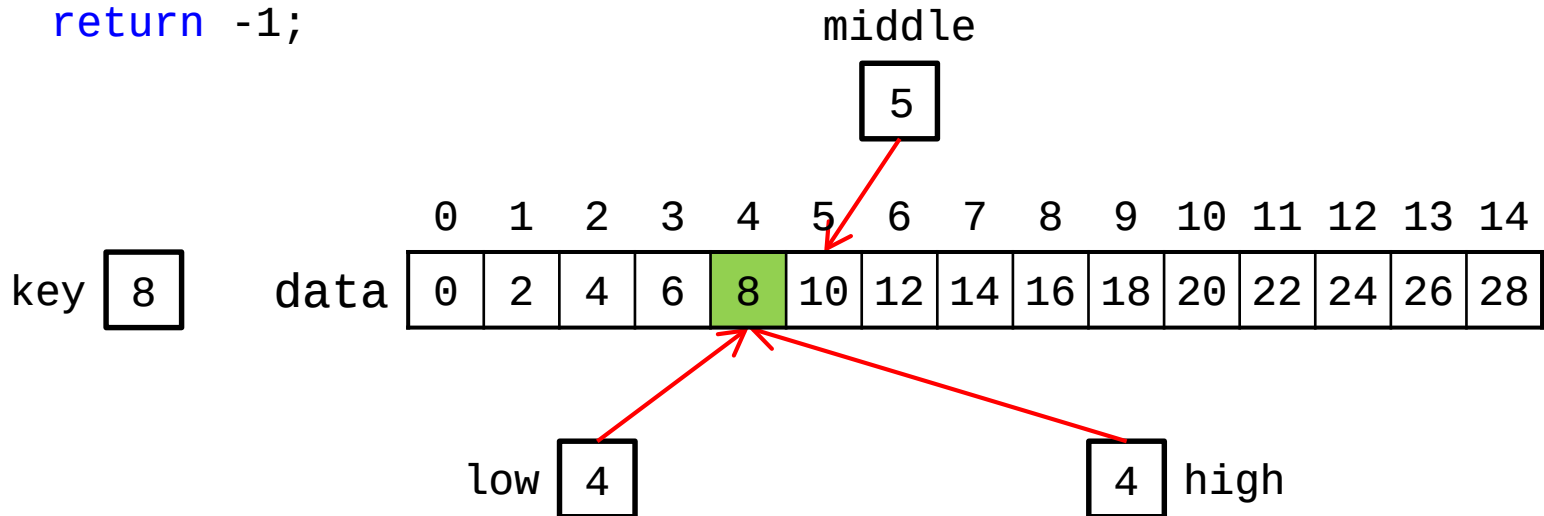
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



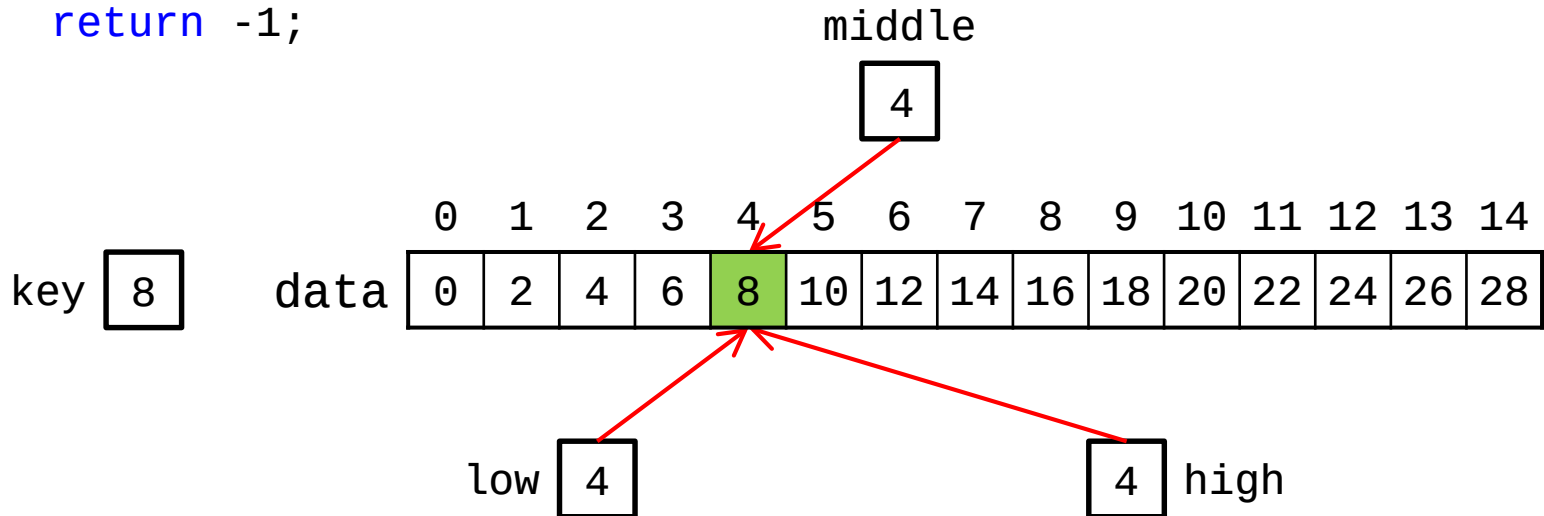

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



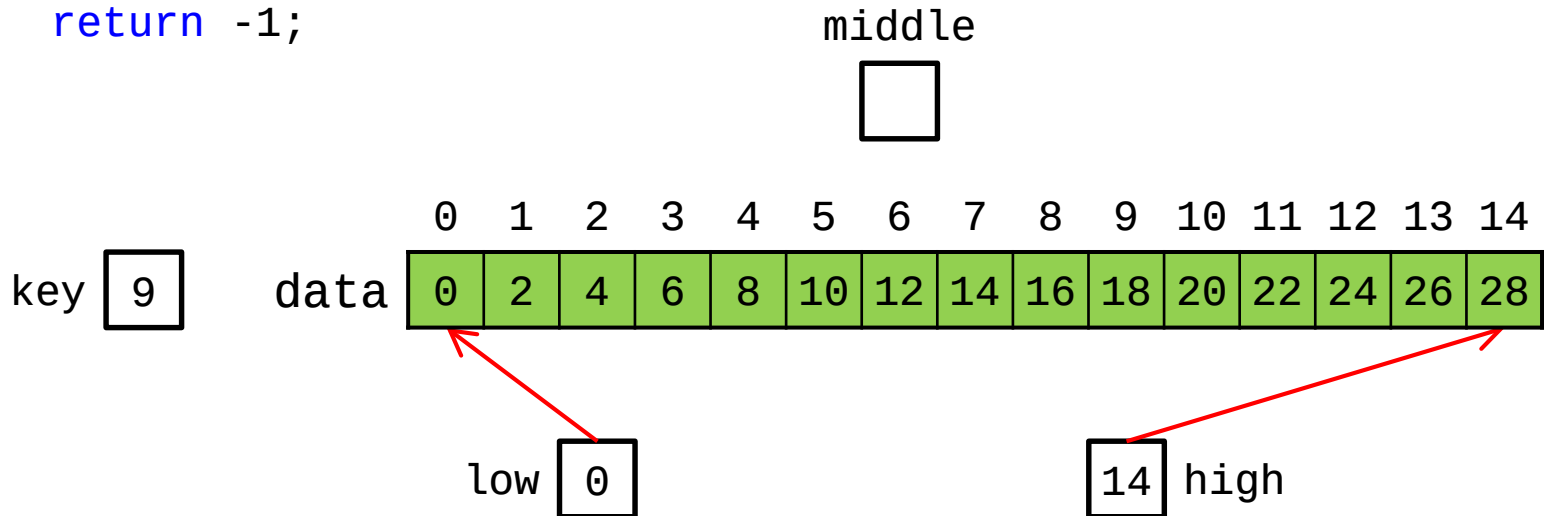

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



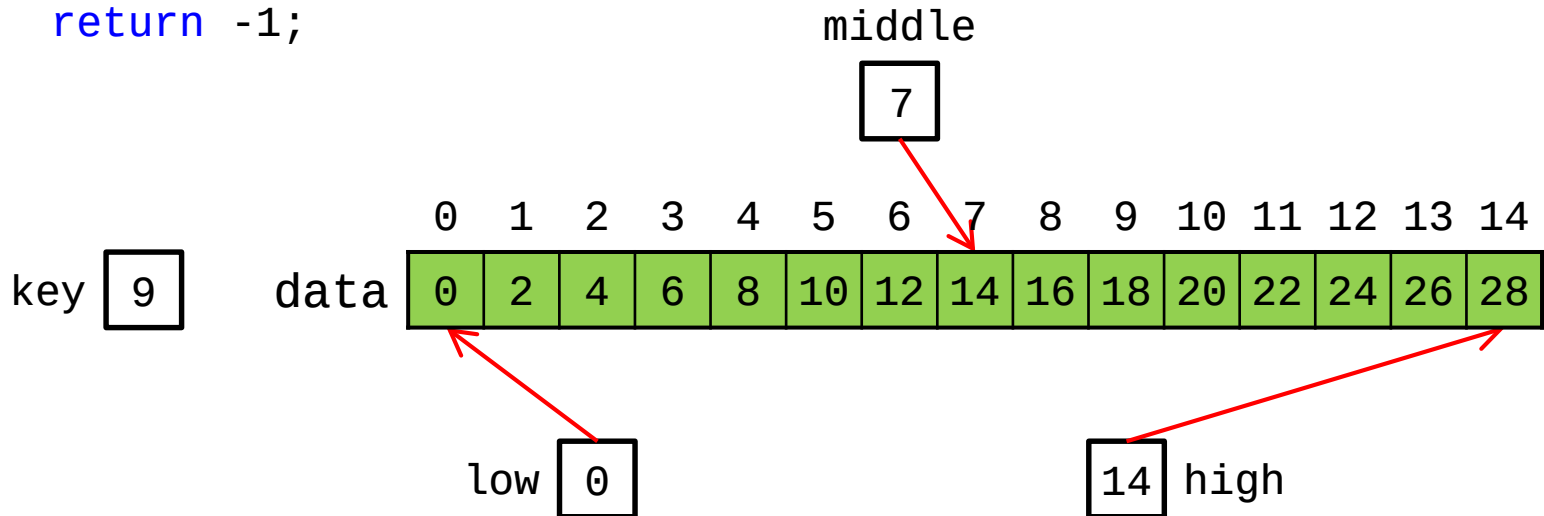
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



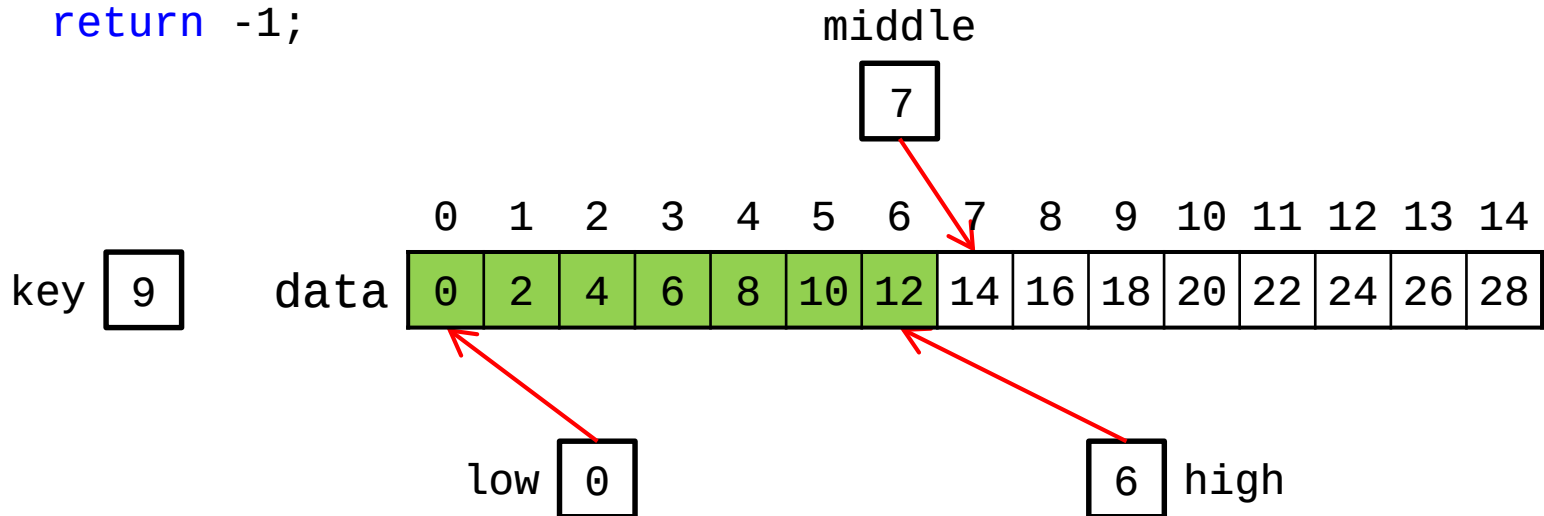
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



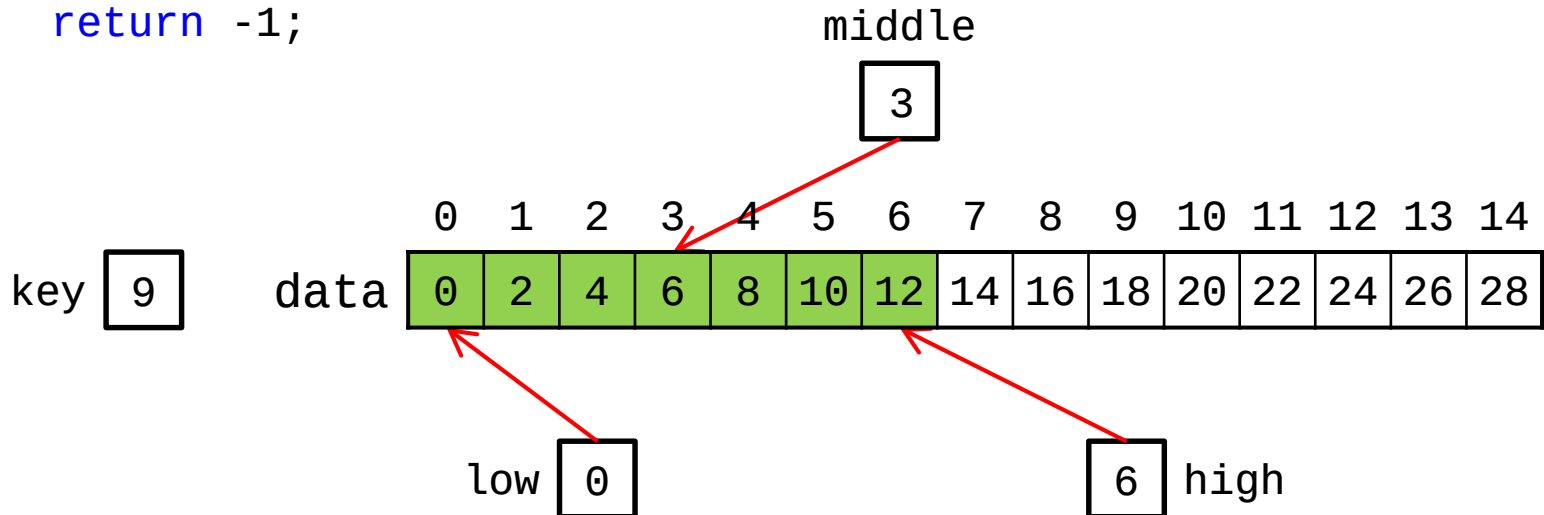
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



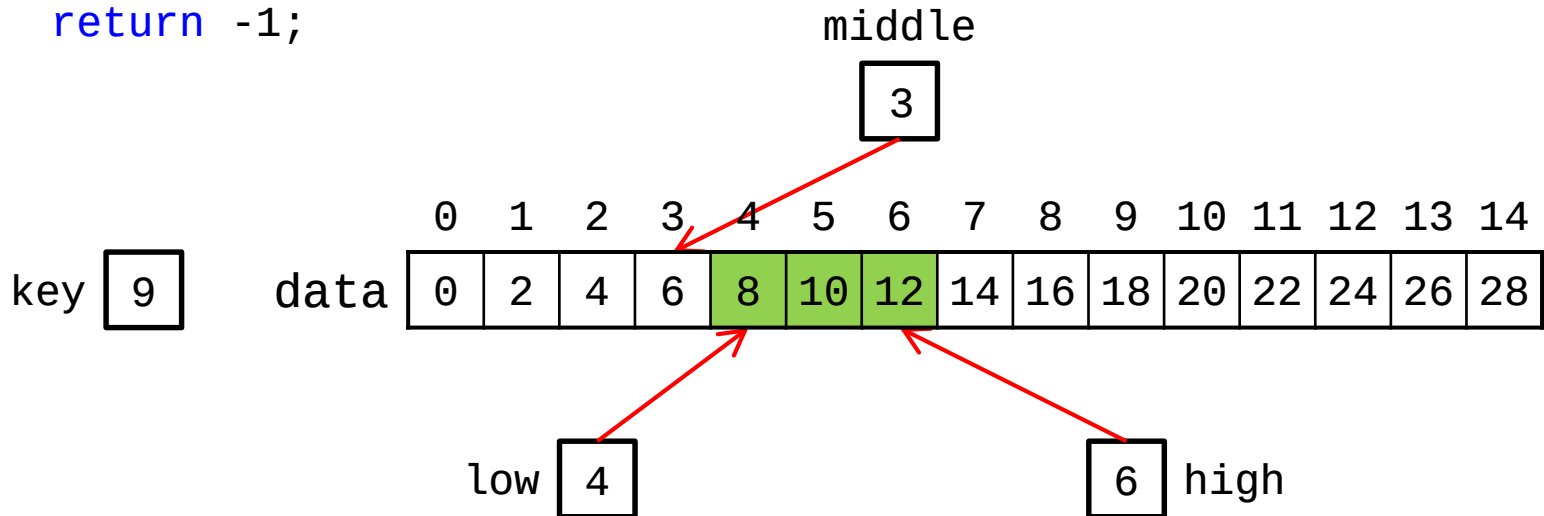
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



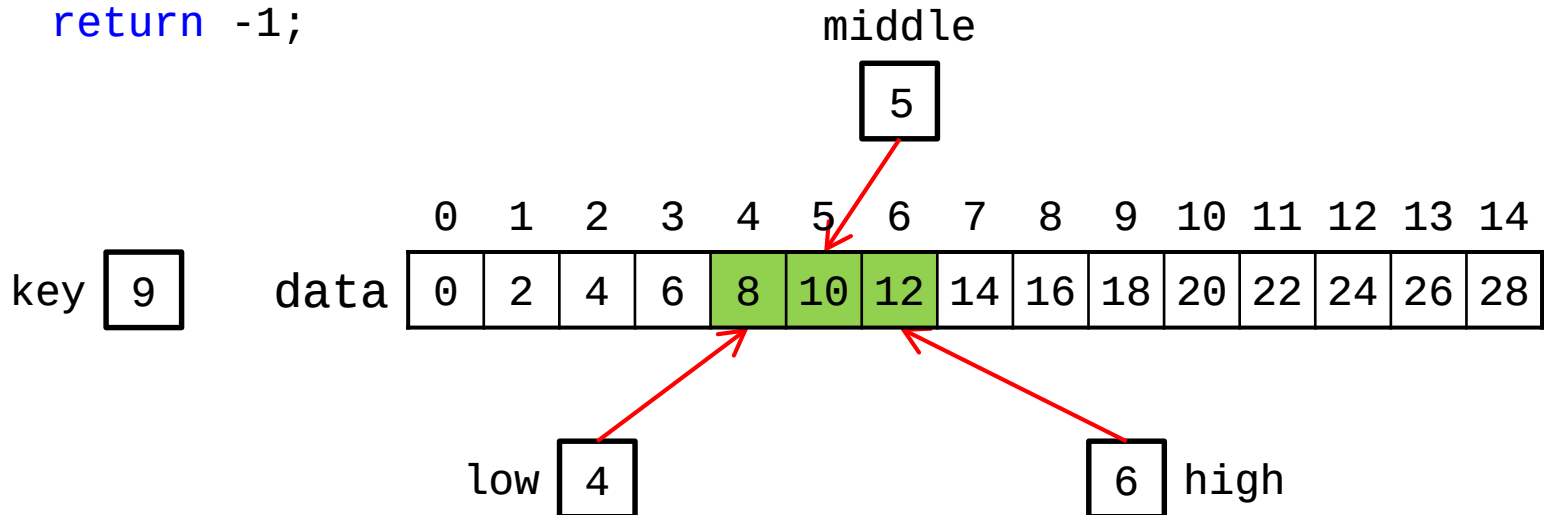
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



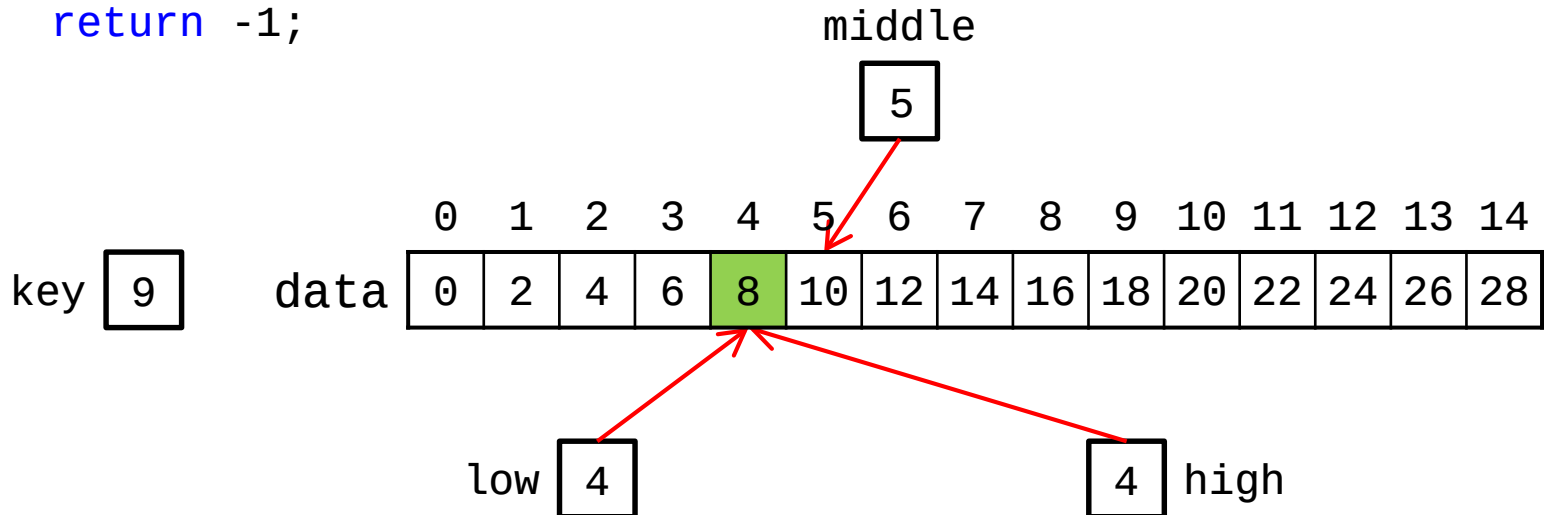

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



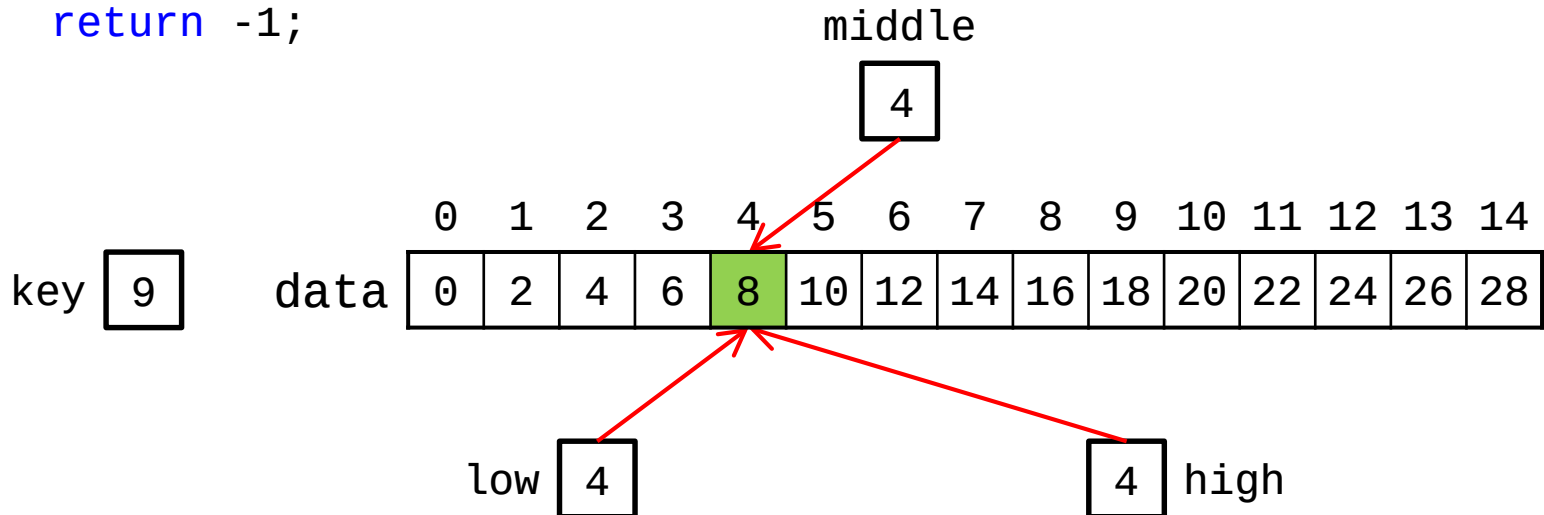
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



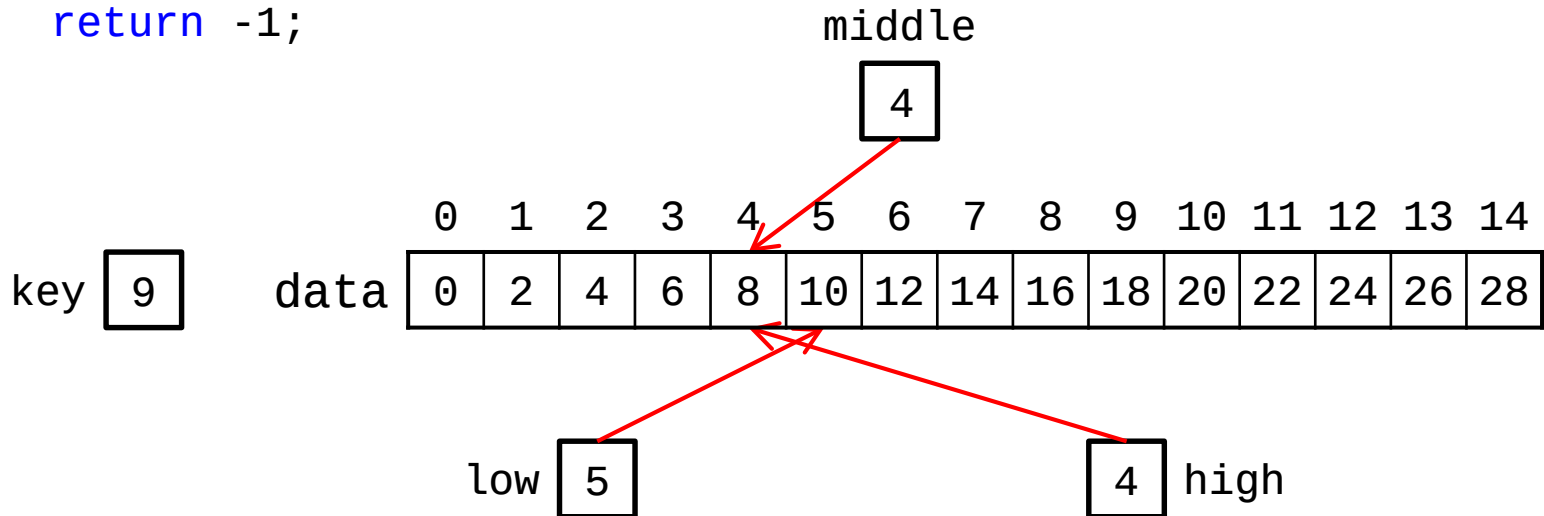
```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```



Recursive Binary Search

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

        int binarySearch( int low, int high )
        {

        }

        return -1;
    }

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    ( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )

        else

    }
    return -1;
}
;
;

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )

        else

    }
    return -1;
}
;
;

```



```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            return binarySearch(                );
        else
            return binarySearch(                );
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```

key

9

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```

bS(0, 14)

key

9

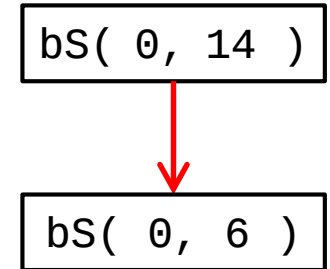
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 9

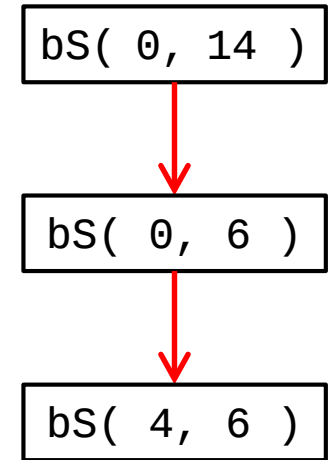
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 9

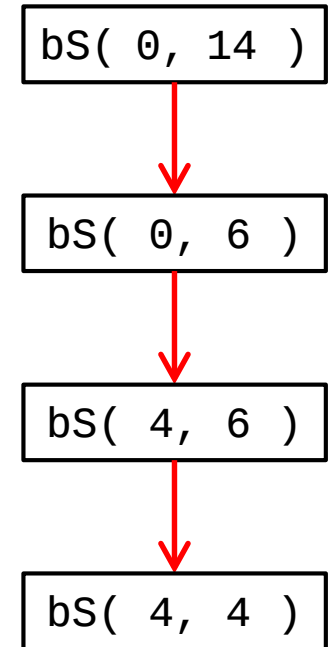
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 9

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

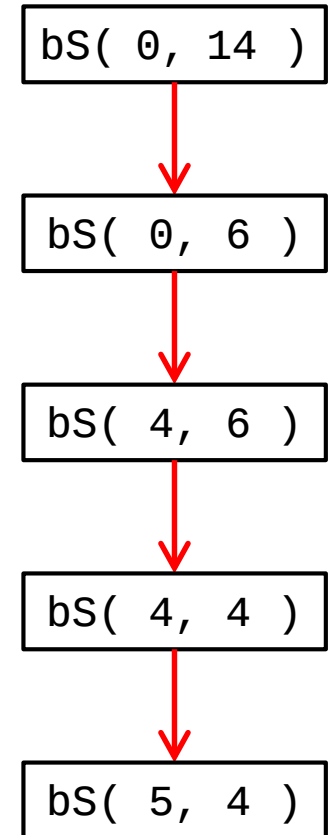
int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```

key 9

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28




```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```

bS(0, 14)

key

8

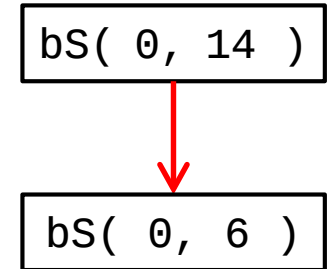
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 8

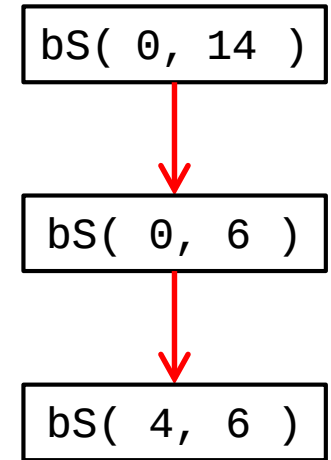
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 8

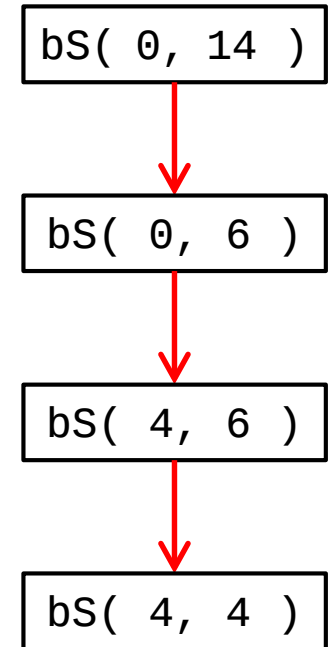
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

```

int binarySearch( int low, int high )
{
    if( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            return binarySearch( low, middle - 1 );
        else
            return binarySearch( middle + 1, high );
    }
    return -1;
}

```



key 8

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
data	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28

Pointer Notation

```
int main()
{
    srand( static_cast< int >( time( 0 ) ) );

    int number = 7;
    data = new int[ number ];

    for( int i = 0; i < number; i++ )
        data[ i ] = 10 + rand() % 90;

    bubbleSort( number );

    cout << "Enter search key: ";
    cin >> key;

    int position = binarySearch( 0, number - 1 );
    if( position != -1 )
        cout << "Found key in position " << position << ".\n\n";
    else
        cout << "Key not found.\n\n";

    delete[] data;
}
```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;

        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return -1;
}

```

```
int main()
{
    srand( static_cast< int >( time( 0 ) ) );

    int number = 7;
    data = new int[ number ];

    for( int *p = data; p < data + number; ++p )
        *p = 10 + rand() % 90;

    bubbleSort( number );

    cout << "Enter search key: ";
    cin >> key;

    int *position = binarySearch( data, data + ( number - 1 ) );
    if( position != nullptr )
        cout << "\nFound key in position " << position - data << ".\n";
    else
        cout << "\nKey not found.\n\n";

    delete[] data;
}
```



```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle =
        if( key ==
            return middle;
        if( key <
            high =
        else
            low =
    }
    return nullptr;
}

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle )
            return middle;
        if( key < *middle )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return nullptr;
}

```

```

int binarySearch( int low, int high )
{
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```
int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle )
            return middle;

        if( key < *middle )
            high = middle - 1;
        else
            low = middle + 1;
    }

    return nullptr;
}
```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```






 position

key 

```

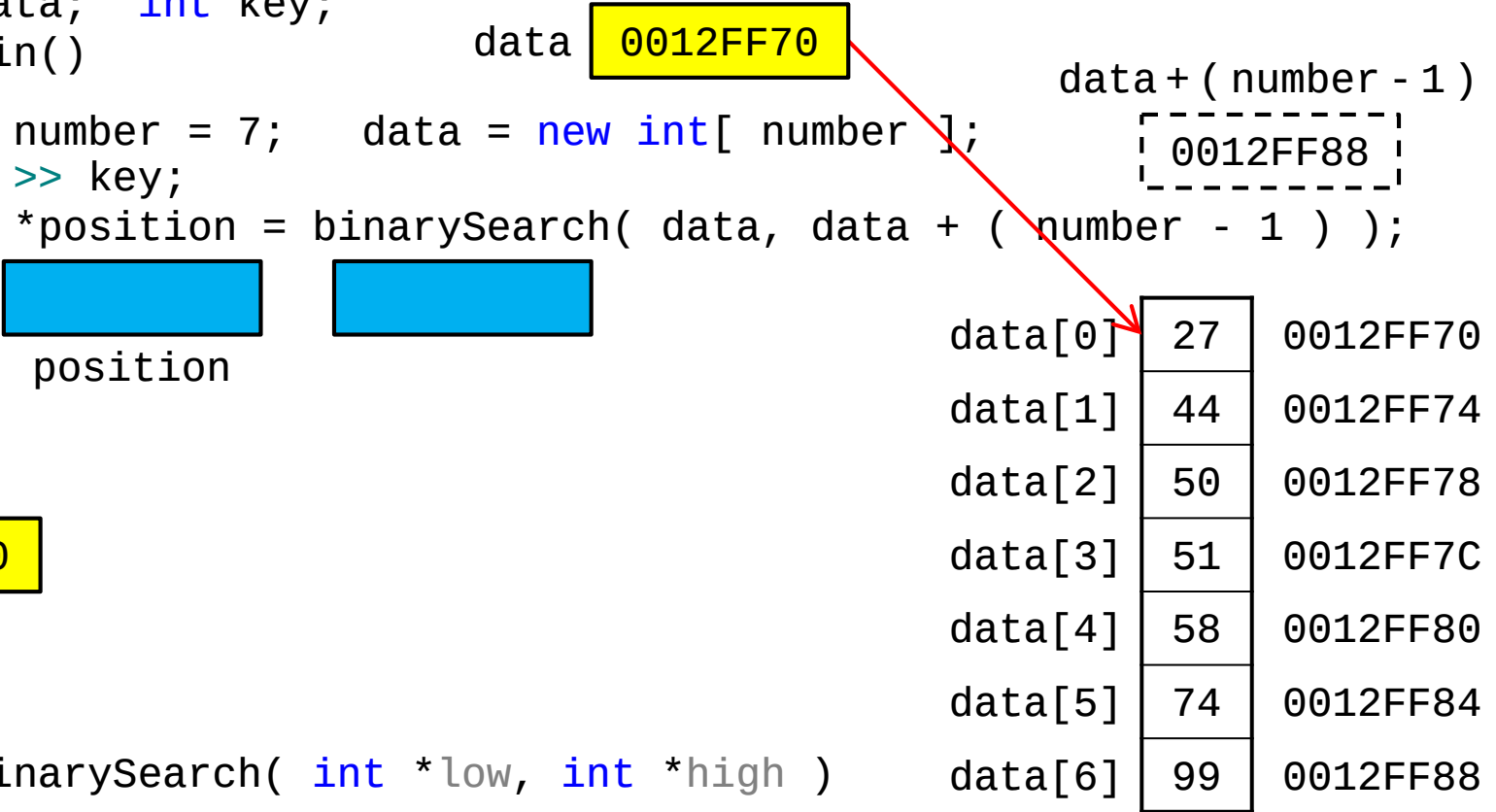
int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```



data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```



```

int *data;  int key;
int main()
{
    int number = 7;  data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

position

key 50

low 0012FF78

middle 0012FF74

high 0012FF78

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;  data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

0012FF78

position

key 50

low 0012FF78
middle 0012FF78
high 0012FF78

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;  data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

0012FF78
0012FF78

position

key 50

low 0012FF78
middle 0012FF78
high 0012FF78

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```



```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```






 position

key 

```

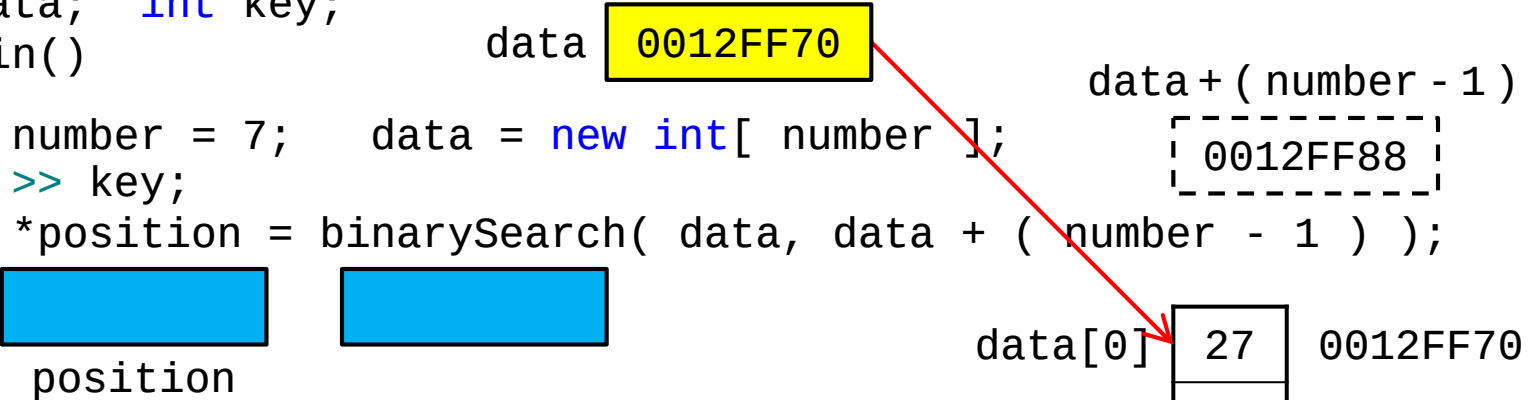
int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```



data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

key 

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```



```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

position

key 47

low 0012FF70
middle 0012FF74
high 0012FF78

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1) 0012FF88

position

key

47

low

0012FF78

middle

0012FF78

high

0012FF78

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```

int *data;  int key;
int main()
{
    int number = 7;  data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

position
0

key 47

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

low 0012FF78
middle 0012FF78
high 0012FF74

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```



```

int *data;  int key;
int main()
{
    int number = 7;    data = new int[ number ];
    cin >> key;
    int *position = binarySearch( data, data + ( number - 1 ) );
}

```

data 0012FF70
data + (number - 1)
0012FF88

0 0
 position

key 47

low 0012FF78
 middle 0012FF78
 high 0012FF74

data[0]	27	0012FF70
data[1]	44	0012FF74
data[2]	50	0012FF78
data[3]	51	0012FF7C
data[4]	58	0012FF80
data[5]	74	0012FF84
data[6]	99	0012FF88

```

int* binarySearch( int *low, int *high )
{
    while( low <= high )
    {
        int *middle = data + ( ( low - data ) + ( high - data ) ) / 2;
        if( key == *middle ) return middle;
        if( key < *middle ) high = middle - 1;
        else low = middle + 1;
    }
    return nullptr;
}

```

```
int binarySearch( int low, int high )
{
    if( low > high )
        return -1;

    int middle = ( low + high ) / 2;
    if( key == data[ middle ] )
        return middle;

    if( key < data[ middle ] )
        return binarySearch( low, middle - 1 );
    else
        return binarySearch( middle + 1, high );
}
```



```
int binarySearch( int low, int high )
{
    if( low > high )
        return -1;

    int middle = ( low + high ) / 2;
    if( key == data[ middle ] )
        return middle;
    if( key < data[ middle ] )
    {
        high = middle - 1;
        while( low <= high )
        {
            int middle = ( low + high ) / 2;
            if( key == data[ middle ] )
                return middle;
            if( key < data[ middle ] )
                high = middle - 1;
            else
                low = middle + 1;
        }
        return -1;
    }
}
```

```
else
{
    low = middle + 1;
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}
}
```

```

if( key == data[ middle ] )
    return middle;
if( key < data[ middle ] )
{
    high = middle - 1;
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```

else
{
    low = middle + 1;
    while( low <= high )
    {
        int middle = ( low + high ) / 2;
        if( key == data[ middle ] )
            return middle;
        if( key < data[ middle ] )
            high = middle - 1;
        else
            low = middle + 1;
    }
    return -1;
}

```

```
int binarySearch( int low, int high )
{
    if( low > high )
        return -1;

    int middle = ( low + high ) / 2;
    if( key == data[ middle ] )
        return middle;
    if( key < data[ middle ] )
    {
        high = middle - 1;
        return binarySearch( low, high );
    }
    else
    {
        low = middle + 1;
        return binarySearch( low, high );
    }
}
```



```
int linearSearch1( int first, int last )  
{
```

```
}
```

```
int linearSearch2( int first, int last )  
{
```

```
}
```

```
int linearSearch1( int first, int last )
{
    if( first == last )
        if( data[ last ] == key )
            return last;
        else
            return -1;

}
```

```
int linearSearch2( int first, int last )
{
    if( first == last )
        if( data[ first ] == key )
            return first;
        else
            return -1;

}
```

```
int linearSearch1( int first, int last )
{
    if( first == last )
        if( data[ last ] == key )
            return last;
        else
            return -1;
    if( data[ last ] == key )
        return
    else
        return
}
```

```
int linearSearch2( int first, int last )
{
    if( first == last )
        if( data[ first ] == key )
            return first;
        else
            return -1;
    if( data[ first ] == key )
        return
    else
        return
}
```

```
int linearSearch1( int first, int last )
{
    if( first == last )
        if( data[ last ] == key )
            return last;
        else
            return -1;
    if( data[ last ] == key )
        return last;
    else
        return linearSearch1( first, last - 1 );
}
```

```
int linearSearch2( int first, int last )
{
    if( first == last )
        if( data[ first ] == key )
            return first;
        else
            return -1;
    if( data[ first ] == key )
        return first;
    else
        return linearSearch2( first + 1, last );
}
```

```
int search1( int k, int f, int l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search1( k, f, l - 1 );
}
```

```
int search2( int k, int f, int l )
{
    if( f == l )
        if( d[f] == k ) return f;
        return -1;
    if( d[f] == k ) return f;
    return search2( k, f + 1, l );
}
```

```

int search( int6 k, int0 f, int2 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );

```

0
0
1

6	9	7
---	---	---

```

int search( int6 k, int0 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```

```

int search( int6 k, int0 f, int1 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );
}

```

0
0
0

```

int search( int k, int f, int l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```

```

int search( int6 k, int0 f, int0 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );
}

```

```

int search( int k, int f, int l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```

```

int search( int9 k, int0 f, int2 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );

```

6
9
7

1
0
1

```

int search( int9 k, int0 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );

```

1
1
2

```

int search( int9 k, int0 f, int1 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );

```

```

int search( int9 k, int1 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );

```

```

int search( int k, int f, int l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );

```

```

int search( int k, int f, int l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );

```

```

int search( int8 k, int0 f, int2 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );
}

```

-1 0 1

6	9	7
---	---	---

```

int search( int8 k, int0 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```

-1 1 2

```

int search( int8 k, int0 f, int1 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );
}

```

-1 0 0

```

int search( int8 k, int1 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```

-1 2 2

```

int search( int8 k, int0 f, int0 l )
{
    if( f == l )
        if( d[l] == k ) return l;
        else return -1;
    if( d[l] == k ) return l;
    return search( k, f, l - 1 );
}

```

```

int search( int8 k, int2 f, int2 l )
{
    if( f == l )
        if( d[f] == k ) return f;
        else return -1;
    if( d[f] == k ) return f;
    return search( k, f + 1, l );
}

```



```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```
int linearSearch( int last )
{
    if( last == -1 )

        if( data[ last ] == key )
            return last;

    return linearSearch( last - 1 );
}
```

```
int linearSearch( int last )
{
    if( last == 0 )
        if( data[ 0 ] == key )
            return 0;
        else
            return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```
int linearSearch( int last )
{
    if( last == -1 )
        return -1;

    if( data[ last ] == key )
        return last;

    return linearSearch( last - 1 );
}
```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```



```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      1

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      1

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      0

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1

```

```

      -1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      1

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      1

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      0

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      -1

```

```

      -1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      1

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      1

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      -1

```

```

      -1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      1

```

6	9	7
---	---	---

```

      8      1
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == 0 )
        if( d[ 0 ] == k ) return 0;
        else return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```

```

      8      2
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      1

```

```

      8      1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      0

```

```

      8      0
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}
      -1      -1

```

```

      -1
int search( int k, int last )
{
    if( last == -1 ) return -1;
    if( d[ last ] == k ) return last;
    return search( k, last - 1 );
}

```